OptiWatt: Smart Energy Management System

Diba Jabin Fariha Tithy[1], Hujaifa Islam Johan[2], Md. Toufiq Imroz Khealid Khan[3],
M. M. Sayem Prodhan[4], Sadid Ahmed[5]
Department of Computer Science and Engineering
United International University, Dhaka, Bangladesh
[1]dtithy2330222@bscse.uiu.ac.bd, [2]mjohan2330149@bscse.uiu.ac.bd, [3]mkhan2330984@bscse.uiu.ac.bd,
[4]mprodhan2330411@bscse.uiu.ac.bd, [5]sahmed2330154@bscse.uiu.ac.bd

***Abstract*—This paper presents OptiWatt, a cost-effective smart energy management system designed for single-room applications that combines real-time energy monitoring with occupancy-aware automation. The system utilizes an ESP32 microcontroller as the central processing unit, integrating PZEM-004T power meters for accurate energy measurement and dual HC-SR04 ultrasonic sensors for occupancy counting. By automatically disconnecting non-critical loads when rooms are unoccupied while allowing manual override through a web interface, OptiWatt achieves a practical balance between energy efficiency and user control. The implemented prototype demonstrates the feasibility of affordable, privacy-preserving residential energy optimization solutions suitable for deployment in developing regions where commercial alternatives remain economically inaccessible.**

***Index Terms*—Smart Energy Management, Occupancy Detection, ESP32, PZEM-004T, IoT, Energy Efficiency, Home Automation**

I. Introduction

Energy efficiency has become increasingly critical as rising electricity demand and environmental concerns drive households toward smarter consumption patterns. Studies indicate that a significant portion of residential electricity consumption results from appliances operating unnecessarily in unoccupied spaces [9]. Although awareness campaigns have attempted to address this behavioral issue, technological intervention remains necessary to achieve sustained improvements without relying solely on user vigilance.

Existing smart home solutions often present significant barriers to widespread adoption. High acquisition costs place commercial systems beyond reach for middle-income households in developing regions. Complex installation procedures requiring professional expertise increase deployment expenses and create dependencies on specialized service providers. Additionally, privacy concerns arise from camera-based occupancy detection systems that capture visual data, creating potential security vulnerabilities and ethical considerations regarding household surveillance.

These limitations create a clear need for affordable, non-intrusive, and user-friendly energy management systems that balance automation with user control while respecting privacy boundaries. OptiWatt addresses this need through a practical prototype that combines readily available components with embedded intelligence, demonstrating that effective energy management need not require expensive infrastructure or compromise user privacy.

The system provides real-time energy monitoring through dedicated power measurement modules, implements occupancy-based load automation using non-imaging sensors, and offers remote control capabilities through a web-based interface accessible from any network-connected device. This paper presents the complete system architecture, component selection rationale, implementation methodology, and operational characteristics of the functional prototype.

The primary contributions of this work include: (1) a low-cost hardware architecture utilizing off-the-shelf components suitable for single-room deployment, (2) a robust occupancy detection algorithm employing dual ultrasonic sensors for directional counting, (3) practical automation logic incorporating fail-safe defaults and user override mechanisms, and (4) a fully functional demonstration booth showcasing real-world applicability and professional enclosure design suitable for residential installation.

II. Related Work

Smart energy management systems have evolved significantly over the past decade, with various approaches addressing residential consumption optimization through different technological strategies.

A. Occupancy Detection Methods

Occupancy detection forms a critical component of context-aware building automation. Kashyap et al. [5] provide a comprehensive review of occupancy detection technologies, categorizing approaches into passive infrared (PIR) sensors, camera-based vision systems, CO_2 concentration monitoring, and ultrasonic ranging. While camera-based systems offer high accuracy and detailed occupancy information, they introduce privacy concerns that limit residential acceptance. PIR sensors provide privacy preservation but cannot distinguish between entry and exit events, preventing accurate occupancy counting. OptiWatt addresses these limitations by employing dual ultrasonic sensors in a directional configuration, enabling entry/exit discrimination while maintaining privacy through non-imaging detection.

B. Non-Intrusive Load Monitoring

Hart [6] pioneered non-intrusive load monitoring (NILM) techniques that disaggregate total household consumption into individual appliance contributions through analysis of

aggregate power signatures. While NILM provides valuable insights without requiring per-appliance metering, the computational complexity and appliance signature training requirements present implementation challenges for cost-sensitive applications. OptiWatt adopts direct per-load metering using dedicated power monitoring modules, trading the elegance of NILM for implementation simplicity and deterministic accuracy suitable for single-room deployments.

C. Smart Home Platforms

Sharif and Al-Turjman [7] survey commercial and research smart home energy management systems, identifying cost, interoperability, and user acceptance as primary adoption barriers. Commercial platforms such as those offered by major technology companies typically require substantial initial investment and ongoing subscription fees, limiting accessibility in price-sensitive markets. Han et al. [8] demonstrate ZigBee-based home energy management integrating infrared remote controls, achieving efficiency improvements through centralized appliance control. However, their approach requires replacing existing infrastructure with compatible devices, increasing deployment costs.

OptiWatt distinguishes itself from existing work by prioritizing affordability through commodity components, preserving privacy through non-imaging sensors, maintaining local operation without cloud dependencies, and providing user authority through comprehensive manual override mechanisms. The system targets the underserved segment of single-room applications in developing regions where commercial solutions remain economically prohibitive.

III. System Architecture and Design

A. Overall Architecture

The OptiWatt system implements a distributed sensing, centralized control architecture as illustrated in Fig. 1. The ESP32 microcontroller serves as the central processing hub, interfacing with multiple sensor modules through standard communication protocols including UART for power monitoring and GPIO for ultrasonic ranging and relay control. The microcontroller hosts an embedded web server that provides user interaction capabilities through any network-connected browser. This modular design facilitates individual component replacement and enables future expansion to multi-room configurations through additional sensor nodes.

B. Physical Implementation

The physical implementation of OptiWatt utilizes a custom-designed booth enclosure that provides safe separation between high-voltage AC distribution components and low-voltage control electronics. Fig. 2 shows the booth with the front panel closed, presenting a professional exterior suitable for demonstration and highlighting the compact form factor achievable through careful component integration.

Fig. 3 reveals the internal component layout with the front panel removed. The organization demonstrates the modular architecture with clearly separated functional blocks: the ESP32 development board mounted centrally for maintenance access, PZEM-004T modules positioned adjacent to their respective load monitoring points to minimize sensing wire length, relay modules with adequate spacing for heat dissipation, and systematic wire routing that maintains electrical safety clearances. The ultrasonic sensors mount in the doorway opening for occupancy detection and are not visible in the interior view.

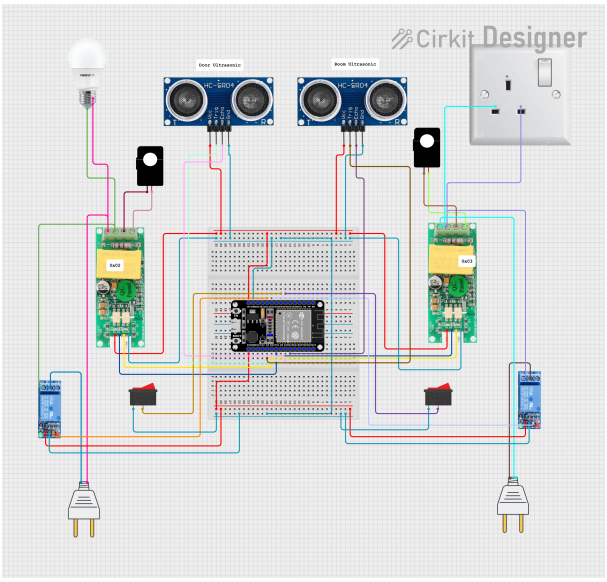

Fig. 1: Complete system architecture showing ESP32 microcontroller connections to ultrasonic sensors, PZEM-004T power monitoring modules, relay control outputs, and network interface.

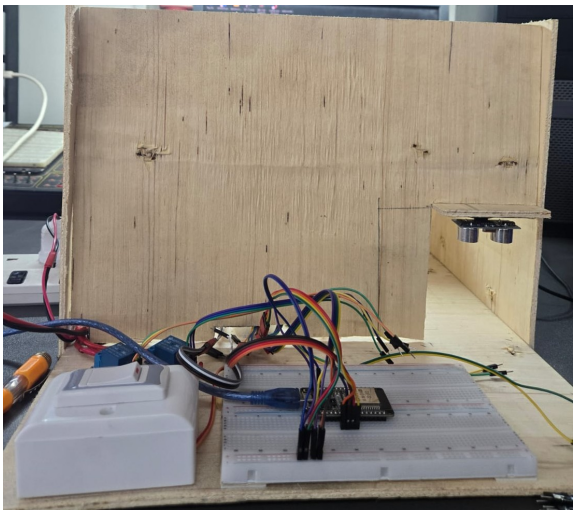

Fig. 2: OptiWatt demonstration booth exterior with closed front panel, showing professional enclosure design suitable for residential installation environments.

C. Key Design Principles

The system design prioritizes three fundamental principles that guide all implementation decisions:

Fig. 3: Internal component layout showing ESP32 microcontroller, PZEM-004T power monitoring modules, relay control boards, and structured wiring maintaining proper separation between high-voltage and low-voltage circuits.

Safety First: All high-voltage AC connections incorporate proper fusing, physical isolation barriers, and electrical clearances according to applicable safety standards. Relay modules include optical isolation to protect low-voltage electronics from potential high-voltage faults. The automation logic implements fail-safe defaults ensuring that critical loads such as refrigeration or security systems cannot be automatically disconnected under any circumstance.

Privacy Preservation: Unlike camera-based occupancy detection systems, OptiWatt employs exclusively non-imaging sensors that measure distance through ultrasonic time-of-flight ranging. This approach eliminates concerns regarding visual surveillance, recorded footage, or personally identifiable information capture, addressing a primary barrier to smart home system acceptance.

User Authority: The automation system functions as an assistance mechanism rather than autonomous controller. Comprehensive manual override capabilities through the web interface ensure users retain ultimate authority over all connected devices. Physical bypass switches provide control redundancy independent of microcontroller operation, preventing user lock-out scenarios during system faults.

IV. Component Descriptions and Specifications

A. ESP32 Development Board

The ESP32 serves as the system's computational core, selected for its optimal balance of processing capability, peripheral availability, and cost efficiency. As shown in Fig. 4,

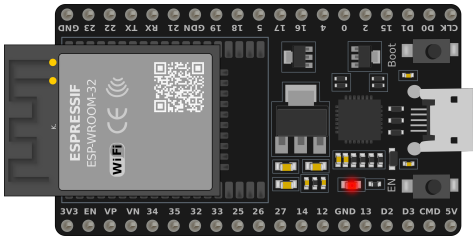

Fig. 4: ESP32 development board serving as the central controller, providing processing, networking, and peripheral interface capabilities.

this dual-core Xtensa LX6 microcontroller integrates IEEE 802.11 b/g/n Wi-Fi and Bluetooth 4.2 connectivity, operating at clock frequencies up to 240 MHz with 520 KB on-chip SRAM [2].

The ESP32's extensive peripheral complement proves essential for OptiWatt's distributed sensing architecture. Dual independent UART interfaces enable simultaneous communication with multiple PZEM-004T modules without bus contention. The 34 programmable GPIO pins accommodate all sensor inputs and relay control signals with sufficient reserve capacity for future expansion. Native FreeRTOS support enables concurrent task management, allowing one processor core to handle time-critical sensor polling and relay state control while the second core manages web server requests and data aggregation.

Integrated Wi-Fi eliminates external communication module requirements, reducing both component count and implementation complexity. The ESP32 hosts a lightweight HTTP server that directly serves the user interface, avoiding dependencies on external cloud services and ensuring continued system operation during internet connectivity interruptions.

Advanced power management features including multiple sleep modes enable potential future battery-backed operation for maintaining occupancy state and configuration during brief power interruptions. The integrated 12-bit ADC channels remain available for future analog sensor integration such as temperature, humidity, or light level monitoring.

B. PZEM-004T V3.0 AC Energy Monitor

The PZEM-004T V3.0, illustrated in Fig. 5, provides accurate single-phase AC power measurement through non-intrusive current sensing combined with direct voltage monitoring. This module represents a significant advancement over discrete component implementations by integrating voltage measurement, current sensing via split-core current transformer, and active power calculation in a compact, pre-calibrated package [3].

The module's internal architecture implements true RMS measurement algorithms, ensuring measurement accuracy with non-sinusoidal loads characteristic of modern switch-mode power supplies and LED lighting. Continuous voltage and current sampling at rates sufficient to capture harmonic content enables calculation of multiple parameters: active power (W),

Fig. 5: PZEM-004T V3.0 power meter module providing non-intrusive energy monitoring through split-core current transformer and integrated voltage sensing.

apparent power (VA), power factor, line frequency (Hz), and cumulative energy consumption (kWh). Communication with the ESP32 occurs via 9600 baud UART using a Modbus-RTU derived protocol with configurable device addresses enabling multiple modules on a shared bus.

The split-core current transformer design allows installation without breaking existing circuits, significantly simplifying deployment and eliminating electrical work requirements. The CT clamps around a single conductor, measuring the magnetic field variations proportional to current flow. This non-intrusive approach maintains electrical safety while enabling accurate measurement across a 0.01A to 100A range suitable for typical residential loads.

Voltage sensing connects in parallel with the AC mains through internal galvanic isolation and voltage division circuitry, measuring from 80V to 260V AC with 0.1V resolution. Power measurement resolution of 0.1W provides sufficient precision for residential monitoring applications where individual loads typically range from tens to hundreds of watts.

Critical to OptiWatt's cost estimation functionality, the PZEM-004T maintains cumulative energy registers in internal non-volatile memory that persists through power cycles. This eliminates requirements for continuous external data logging and ensures billing calculations remain accurate following system power interruptions.

C. HC-SR04 Ultrasonic Sensors

OptiWatt employs a pair of HC-SR04 ultrasonic ranging sensors (Fig. 6) positioned across the doorway entrance to implement directional occupancy counting. This dual-sensor approach enables the system to distinguish between entry and exit trajectories, maintaining an accurate occupancy count without requiring individual identification [4].

Each HC-SR04 module operates by emitting 40 kHz ultrasonic pulses through a piezoelectric transmitter transducer

Fig. 6: HC-SR04 ultrasonic sensors deployed in doorway configuration for directional occupancy counting, enabling entry/exit discrimination.

when triggered by a brief GPIO pulse from the microcontroller. The receiver transducer detects reflected echoes from objects within the beam path, and internal timing circuitry generates an output pulse whose duration corresponds to the acoustic round-trip time. The ESP32 measures this pulse width using hardware timer peripherals and calculates distance using the relationship:

$$d = \frac{t \cdot v_{\text{sound}}}{2}$$

where d represents distance in centimeters, t denotes pulse width in microseconds, and $v_{\text{sound}} \approx 343$ m/s represents the speed of sound in air at standard temperature (20°C).

The doorway counting algorithm implements a finite state machine monitoring activation sequences across the sensor pair. Positioning sensor A at the outer threshold and sensor B at the inner threshold creates a detection zone spanning the doorway width. When an individual crosses the threshold, triggering sensor A before sensor B within a defined temporal window indicates entry direction, incrementing the occupancy counter. The reverse sequence (B before A) indicates exit direction, decrementing the counter. The algorithm incorporates debouncing logic and minimum crossing time thresholds to filter spurious triggers from acoustic noise or simultaneous bi-directional passages.

The effective measurement range of 2 cm to 400 cm with typical accuracy of ± 3 mm suits doorway deployment requirements. The nominal 15-degree beam angle provides adequate coverage width for standard residential doorway dimensions. Operating voltage of 5V DC with peak current consumption below 15 mA imposes minimal power requirements on the system supply.

D. Relay Modules

Electromechanical relay modules (Fig. 7) provide the interface between low-voltage microcontroller control signals and high-voltage appliance switching. OptiWatt employs single-pole, single-throw (SPST) relay modules rated for 10A at 250V AC, providing adequate safety margin for typical residential lighting and small appliance loads.

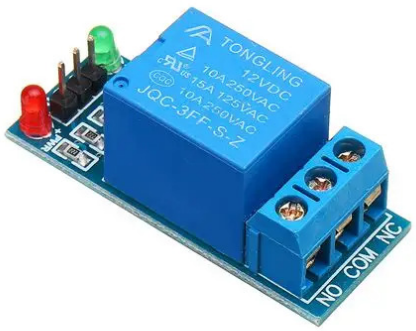

Fig. 7: Optically isolated relay module for safe high-voltage appliance control, showing terminal blocks for AC connections and low-voltage control input.

Each module incorporates several critical safety features. Optical isolation separates the ESP32's 3.3V logic domain from the relay coil circuit, preventing voltage transients or ground faults from propagating to the microcontroller. A flyback diode connected across the relay coil suppresses inductive voltage spikes during coil de-energization. Visual LED indicators provide immediate confirmation of relay state for troubleshooting and system status verification.

The relay coil operates from 5V DC with approximately 70 mA activation current, easily driven by the ESP32's GPIO pins through the optical isolator buffer. Switching time remains below 10 ms, imperceptible to users during normal operation. The contact ratings of 10A resistive load at 250V AC accommodate typical residential lighting fixtures (1–2A), ventilation fans (1–3A), and small appliances within the intended application scope.

For OptiWatt deployment, normally-open (NO) contacts provide the switching function, ensuring fail-safe behavior: in the event of system malfunction, power supply failure, or microcontroller reset, relays de-energize and disconnect controlled loads. Critical loads requiring uninterrupted power such as refrigerators or security systems bypass relay control entirely, connecting directly to mains distribution.

Screw terminal blocks provide strain relief and clearly labeled connection points for AC mains wiring. The module form factor supports potential DIN-rail mounting in future enclosure designs, facilitating professional installation within residential electrical distribution panels.

E. Power Supply and Safety Infrastructure

The system employs a dual-voltage power architecture supplying 5V for sensors and relay modules and 3.3V for the ESP32 logic circuits. A regulated 5V/2A wall adapter provides primary power from mains AC, with an onboard linear regulator stepping down to 3.3V for the microcontroller. Current limiting and overcurrent protection through fusing guard against short circuit conditions.

All high-voltage AC connections reside in a physically separated enclosure compartment with appropriate creepage distances (minimum 4 mm for 250V applications according to electrical safety standards). Screw terminal blocks provide secure wire termination with strain relief. A main circuit breaker or fuse (typically rated 10A) protects the entire distribution circuit, while individual fuses on relay outputs provide per-load fault protection.

Ground connections implement a star topology to minimize ground loop formation and potential noise injection into low-voltage circuits. All exposed metal enclosure components bond to protective earth ground. Cable entry points utilize appropriate grommets or cable glands to maintain insulation integrity and prevent abrasion of conductor insulation.

V. System Operation and Control Algorithms

A. Occupancy Detection Algorithm

The occupancy detection subsystem implements a finite state machine with four primary states: VACANT, OCCUPIED, ENTRY_ARMED, and EXIT_ARMED, as illustrated in the flowchart of Fig. 8. State transitions occur based on ultrasonic sensor activation sequences analyzed through pattern recognition logic.

The algorithm maintains an integer occupancy counter initialized to zero at system startup. Ultrasonic sensors continuously monitor for threshold crossings, defined as measured distances falling below predefined setpoints within specified temporal constraints. When Sensor A (positioned at the door outer edge) triggers followed by Sensor B (positioned at the door inner edge) within a 2-second window, the system interprets this sequence as an entry event and increments the occupancy counter. The reverse sequence (B followed by A within the temporal window) represents an exit event, decrementing the counter. The counter employs floor clamping at zero to prevent negative values that would result from exit detections without corresponding prior entries.

To improve detection robustness against acoustic noise and spurious reflections, the system employs median filtering over a sliding window of 9 samples per sensor. This non-linear filtering approach effectively suppresses isolated outliers while preserving rapid response to genuine occupancy events. A configurable debounce period of 300 ms prevents multiple triggers from a single crossing event. A refractory period of 1.2 seconds after each confirmed occupancy event blocks immediate re-triggering, accommodating the natural pace of human movement through doorways while preventing double-counting.

The EXIT_ARMED state introduces a configurable vacancy delay (default 3 seconds) before triggering load disconnection. This grace period accommodates brief exits without unnecessarily cycling connected loads. If occupancy detection occurs during this delay period, the system immediately transitions to OCCUPIED state and cancels the pending power cutoff action.

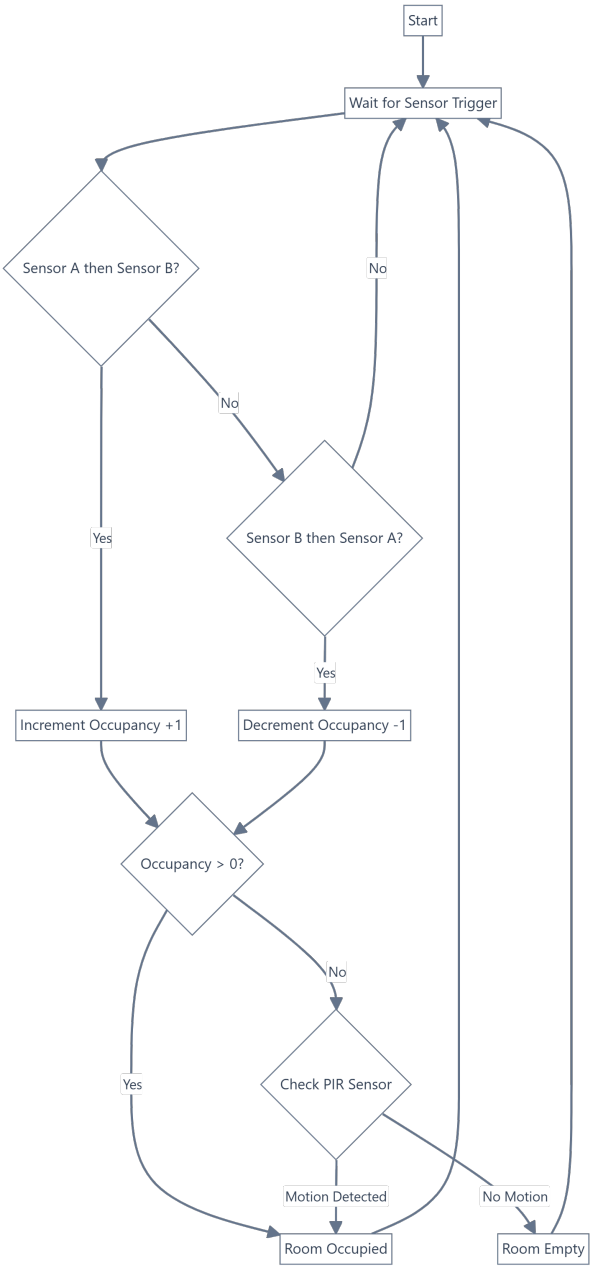

Fig. 8: State machine flowchart for occupancy detection and counting using dual ultrasonic sensors, showing state transitions based on sensor activation sequences.

B. Automated Power Control

The power control subsystem, depicted in the flowchart of Fig. 9, manages relay states based on occupancy status, global automation enable state, and per-device automation preferences. The system classifies each connected load as either critical (exempt from all automation) or non-critical (subject to occupancy-based control when automation is enabled).

When the system transitions to VACANT state following expiration of the vacancy delay period, the automation routine evaluates each non-critical load. If global automation is

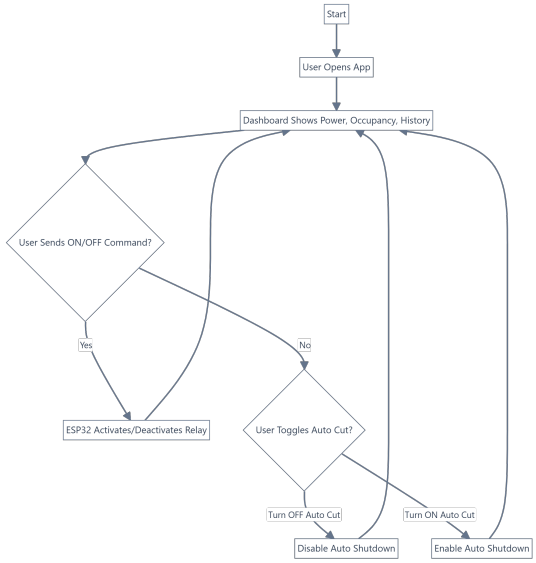

Fig. 9: Automated power control flowchart showing decision logic for relay control based on occupancy state, automation enable flags, and manual override status.

enabled, per-device automation is enabled for that specific load, and no manual override flag is set, the corresponding relay de-energizes, disconnecting the load. The system logs this automation event with timestamp information to support subsequent energy consumption analysis.

Upon return to OCCUPIED state, the system evaluates user preferences stored in non-volatile flash memory. Loads configured for automatic restoration upon occupancy detection re-energize immediately, while loads configured for manual-only restoration remain disconnected until explicit user action through the web interface. This configurable behavior allows users to customize automation aggressiveness matching their specific usage patterns and preferences.

Manual override flags persist until explicitly cleared through user interface action or until the system completes a full vacancy-occupancy-vacancy state cycle. This persistence prevents automation from repeatedly overriding user intentions during temporary absences or intentional load disconnection for maintenance purposes.

C. Energy Monitoring and Data Management

The energy monitoring process, illustrated in Fig. 10, demonstrates the ESP32's periodic polling of each PZEM-004T module at 2 Hz (500 ms intervals) using the Modbus-RTU protocol. Each query transaction returns seven parameters: voltage (V), current (A), active power (W), cumulative energy (kWh), line frequency (Hz), power factor, and alarm status flags. The system applies a 5-sample moving average filter to voltage, current, and power readings to reduce display noise from measurement quantization while maintaining adequate responsiveness to load changes.

Energy values accumulate continuously within the PZEM module's internal non-volatile memory. The ESP32 periodi-

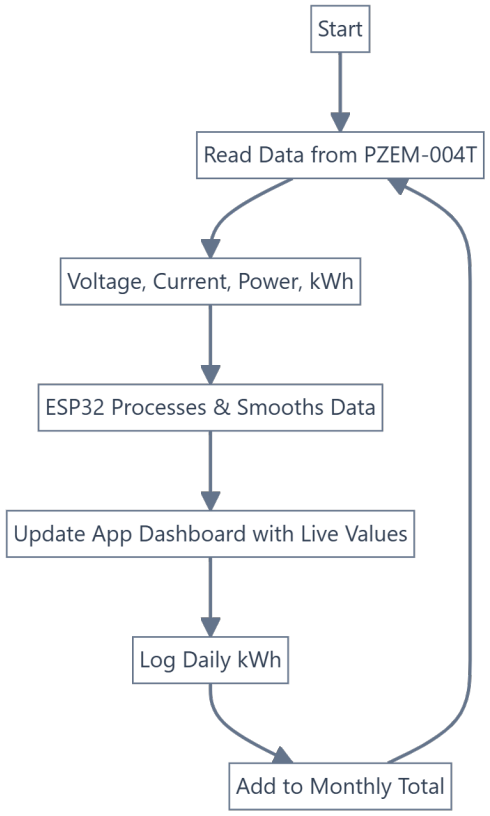

Fig. 10: Real-time energy monitoring flowchart showing PZEM-004T polling sequence, data acquisition, and processing pipeline for display and analysis.

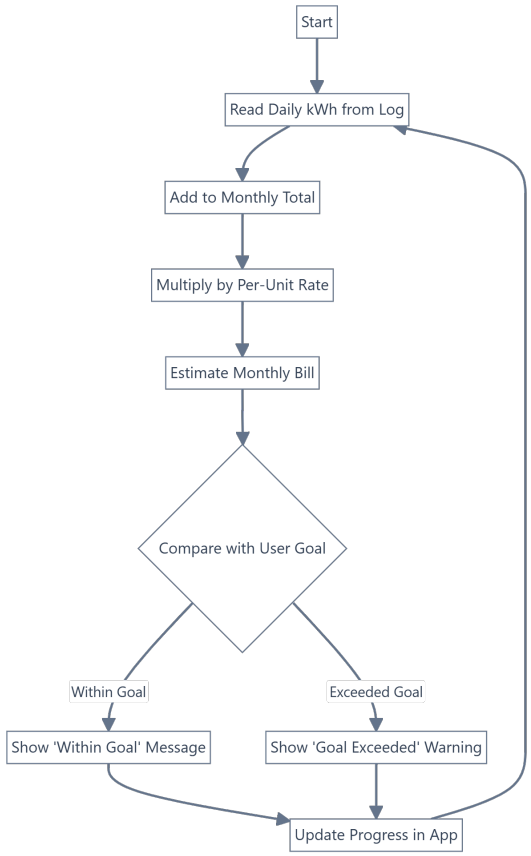

Fig. 11: Monthly cost estimation flowchart implementing slab-based tariff calculation for accurate billing projection based on extrapolated consumption patterns.

cally reads and stores these values in its own flash-based non-volatile storage using the Preferences library to enable calculation of daily and monthly consumption statistics. Timestamps associate each reading with its acquisition time, supporting historical consumption analysis and trend identification.

A background task executing at one-minute intervals computes derived metrics including: daily energy consumption (kWh) relative to stored baseline values, estimated cost based on user-configured tariff rates, and projected monthly consumption by extrapolating current daily consumption averages. These calculations support the predictive features of the web interface, enabling users to anticipate billing amounts and adjust consumption behavior proactively.

D. Monthly Cost Estimation

Fig. 11 presents the monthly cost estimation algorithm that utilizes accumulated energy data to project billing amounts. The system implements a slab-based pricing structure matching Bangladesh residential electricity tariffs, which employ progressive rates increasing with consumption levels.

The algorithm establishes baseline energy readings at the beginning of each calendar month. Throughout the month, it calculates month-to-date (MTD) consumption by subtracting baseline values from current readings. Based on the number of days elapsed in the current month, it computes average daily consumption and extrapolates to estimate total monthly usage.

The slab-based cost calculation applies Bangladesh residential electricity tariffs (as of 2025):

- 0–50 kWh: BDT 4.19/kWh
- 51–75 kWh: BDT 5.72/kWh
- 76–200 kWh: BDT 6.00/kWh
- 201–300 kWh: BDT 6.34/kWh
- 301–500 kWh: BDT 9.94/kWh
- Above 500 kWh: BDT 11.46/kWh

This projection helps users anticipate monthly billing amounts and adjust consumption behavior accordingly. The system updates projections daily as new consumption data becomes available, improving projection accuracy as the month progresses and reducing uncertainty from early-month extrapolations.

E. Web Interface and Remote Control

The ESP32 hosts a lightweight HTTP server that delivers a responsive single-page web application compatible with desktop and mobile browsers. The interface, shown in Fig. 12, provides real-time status visualization and interactive control

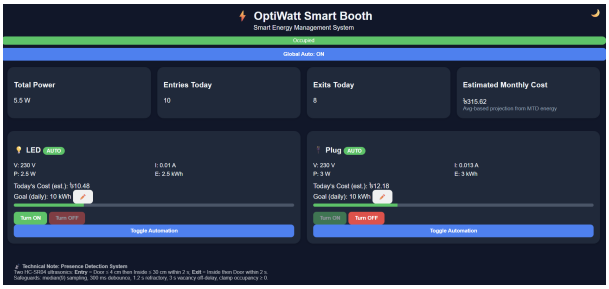

Fig. 12: Web interface showing real-time energy consumption dashboard and device control panel with responsive design supporting desktop and mobile access.

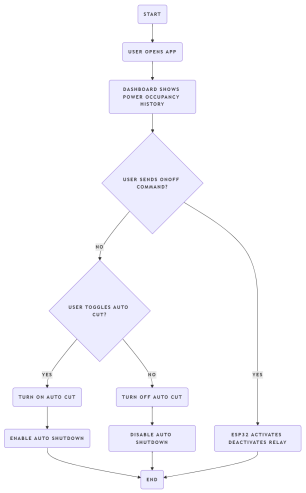

Fig. 13: Web application control flow and manual override logic showing user interaction paths and system response sequences.

capabilities without requiring dedicated mobile applications or platform-specific software installation.

The manual control flow, detailed in Fig. 13, demonstrates user interaction patterns with the system through the web interface. The interface updates via periodic asynchronous JavaScript (AJAX) requests: 1 Hz for occupancy status and system state, 2 Hz for power readings during active monitoring. Users can toggle individual relay states, enable or disable automation per load, adjust vacancy delay timers, and configure cost estimation parameters including daily consumption goals and electricity tariff rates.

All control actions immediately update relay states and set appropriate override flags to prevent automation from conflicting with explicit user commands. The interface provides visual feedback through color-coded status indicators, animated state transitions, and confirmation messages. Historical data visualization allows users to review daily consumption patterns and identify opportunities for waste reduction.

Authentication via configurable password prevents unauthorized control access, while the local-only Wi-Fi access point architecture eliminates external attack surfaces common in cloud-connected systems. The interface maintains full functionality during internet outages, operating entirely within the local network without external dependencies.

TABLE I: System Cost Breakdown

Component	Qty	Unit (BDT)	Total (BDT)
ESP32 DevKit	1	375	375
PZEM-004T V3.0	2	3,200	6,400
HC-SR04 Ultrasonic	2	75	150
Relay Modules	2	85	170
Power Supply & Misc.	–	–	1,500
Total			**8,595**

VI. Cost Analysis

Table I presents the complete bill of materials for the OptiWatt prototype based on component prices in the Dhaka, Bangladesh market. The total component cost of BDT 8,595 represents a significant cost advantage compared to commercial smart energy management systems, which typically range from BDT 15,000 to 25,000 per room in the local market. The modular architecture enables further cost reduction in multi-room deployments through shared components, with a single ESP32 capable of managing multiple rooms via additional sensors and relay modules.

Operational costs remain minimal, with control electronics consuming approximately 2W continuously (17.5 kWh monthly), adding approximately BDT 70–90 to monthly electricity bills depending on the household's consumption tier. The primary value proposition extends beyond direct financial savings to encompass several additional benefits:

Behavioral awareness: Real-time consumption monitoring makes energy usage visible and quantifiable, encouraging conservation behaviors that may compound savings over time through increased user consciousness of consumption patterns.

Billing transparency: Continuous consumption verification provides confidence in utility billing accuracy, addressing common concerns about meter reading errors or estimated billing in regions where automated meter reading infrastructure may be limited.

Convenience automation: Automated control eliminates repetitive manual switching tasks, offering particular value for hard-to-reach switches, users with physical mobility limitations, or households with irregular occupancy patterns.

Educational platform: The system serves as a practical learning platform for understanding energy consumption patterns, experimenting with efficiency strategies, and developing awareness of the relationship between appliance operation and electricity costs.

Multi-room deployment scenarios improve economic viability through increased absolute savings potential from higher total consumption and component cost sharing. A single ESP32 microcontroller can coordinate up to six rooms with appropriate sensor and relay expansion, reducing per-room cost to approximately BDT 6,000 for the second and subsequent rooms.

VII. Discussion

A. Strengths and Contributions

OptiWatt successfully demonstrates several key strengths that address identified gaps in existing residential energy management solutions available in developing regions:

Economic accessibility: The implementation cost under BDT 10,000 makes the technology accessible to middle-income households in developing regions where commercial solutions remain prohibitively expensive. Component costs represent approximately one-third to one-half those of comparable commercial systems while providing equivalent core functionality for single-room applications.

Privacy preservation: The non-imaging sensor approach completely eliminates ethical and security concerns associated with camera-based occupancy detection systems. Users can operate the system with confidence that no visual data, personally identifiable information, or behavioral patterns beyond simple room occupancy are captured, stored, or potentially compromised through security breaches.

User control retention: Comprehensive manual override capabilities through the web interface ensure users maintain ultimate authority over automation decisions. The system functions as an assistance mechanism rather than autonomous controller, building trust and acceptance by respecting user preferences and allowing immediate intervention when automation behavior does not align with immediate needs.

Modular architecture: The clear separation of sensing, control, and actuation functions facilitates customization and future expansion. Individual components can be upgraded or replaced without system-wide redesign. The architecture naturally extends to multi-room configurations through additional sensor nodes communicating with a central coordinator.

Local operation independence: The elimination of cloud service dependencies ensures continued operation during internet outages, avoids recurring subscription fees, and prevents vendor lock-in scenarios where system functionality depends on continued third-party service provision. All data remains on-premises under direct user control.

The dual ultrasonic sensor approach for occupancy detection represents a practical compromise between accuracy and cost. While not achieving the precision of sophisticated computer vision systems or multi-sensor fusion approaches requiring extensive calibration and training, it provides sufficient reliability for the intended automation use case at a fraction of the implementation cost and complexity.

B. Limitations and Challenges

Several limitations warrant acknowledgment and suggest directions for future improvement:

Single-room scope: The current implementation monitors and controls only one room. Practical residential deployment requires coordinated multi-room systems with centralized oversight and whole-home energy budgeting capabilities. Scaling introduces challenges in sensor network coordination, distributed state management, and comprehensive consumption tracking across multiple spaces.

Occupancy counting accuracy: The dual ultrasonic approach experiences detection failures during simultaneous doorway crossings by multiple individuals or very rapid passages exceeding 2 m/s transit speed. While these represent edge cases in typical residential usage, high-traffic environments or households with multiple simultaneous occupants may experience more frequent count errors. The 3-second vacancy delay provides some tolerance for brief counting errors but cannot fully compensate for systematic undercounting in high-traffic scenarios.

Sensor calibration sensitivity: Ultrasonic sensors require careful physical positioning and threshold tuning to achieve reliable operation in specific doorway configurations. Environmental factors including temperature variations affecting sound velocity, acoustic reflections from nearby furniture or wall surfaces, and doorway geometry irregularities can necessitate recalibration. Installation by non-technical users may prove challenging without detailed guidance and troubleshooting support.

Installation complexity: Despite the modular design intent, proper installation requires basic electrical knowledge for safe high-voltage connections. Terminal block wiring, appropriate fuse selection, and proper ground bonding follow electrical standards that may be unfamiliar to typical homeowners. Professional installation assistance would increase total deployment cost, potentially offsetting some of the economic advantage over commercial systems with professional installation included.

Physical switch bypass: The current implementation allows physical wall switches to override system control. While this provides important fail-safe functionality, it creates potential confusion when automation and manual control conflict. Users must understand the interaction between physical switches and web-based control to avoid unexpected behavior.

Limited load types: The relay-based switching architecture suits resistive loads (incandescent lighting, heating elements) and small motor loads (fans) but may require modification for specialized loads such as dimmable LED fixtures, electronic ballast fluorescent lighting, or appliances with soft-start requirements.

VIII. Future Work and Extensions

A. Multi-Room Integration

The most immediately valuable enhancement involves extending the system to multiple rooms with centralized coordination. This requires:

Distributed sensing architecture: Each room would host a sensor node comprising an ESP32 with local ultrasonic sensors and relay control outputs that monitor local occupancy and control local loads. Room nodes would communicate with a central coordinator using the ESP-NOW protocol, which provides low-latency peer-to-peer communication without Wi-Fi infrastructure overhead or access point association delays.

Whole-home energy budgeting: The central coordinator would aggregate energy data across all rooms, implement priority-based load shedding during peak demand periods to

avoid exceeding demand targets, and optimize consumption to maintain user-defined budget constraints or participate in utility demand response programs offering time-of-use pricing incentives.

Inter-room context awareness: The system could implement intelligent rules incorporating multi-room context, such as "maintain bedroom lighting availability during nighttime hours regardless of momentary vacancy" or "provide living room climate control priority when multiple rooms show occupancy," enabling context-sensitive automation beyond simple per-room logic.

Unified dashboard: A centralized web interface would display whole-home consumption alongside per-room breakdowns, enabling users to identify high-consumption areas, compare room-to-room efficiency, and allocate consumption reduction efforts to areas with greatest impact potential.

B. Enhanced Hardware Design

Future hardware iterations should address identified limitations and improve deployment reliability:

Custom PCB design: Transitioning from breadboard and development board prototypes to purpose-designed printed circuit boards would improve reliability through proper trace routing, controlled impedance design, adequate ground plane implementation, and integrated component mounting. Manufacturing costs would decrease significantly with volume production, potentially reducing total system cost below BDT 7,000 for quantities exceeding 100 units.

Professional enclosures: DIN-rail mountable enclosures with clear component labeling, proper cable glands for wire entry, and transparent covers for status indicator LEDs would support professional installation within residential electrical distribution panels. Separate compartments maintaining physical barriers between high-voltage and low-voltage circuits would enhance safety and facilitate electrical inspection approval.

Battery backup capability: Incorporating a small lithium battery (18650 cell providing approximately 3000 mAh capacity) would maintain occupancy state, system configuration, and energy baseline data during brief power outages. This prevents automation state confusion when power restoration occurs and ensures energy consumption tracking continuity across power interruption events.

Additional sensing modalities: Integration of temperature, humidity, and ambient light sensors would enable more sophisticated automation rules such as "disable fan automation if room temperature exceeds 30°C" or "reduce lighting automation during daylight hours when natural illumination is sufficient." These sensors add minimal cost (typically BDT 50–150 each) while significantly enhancing automation capability and user value.

Smart circuit breaker integration: Replacing separate PZEM modules and relay assemblies with integrated smart circuit breakers would simplify installation through direct electrical panel integration, improve reliability by eliminating external wiring between monitoring and switching functions, and reduce component count. However, this approach increases per-circuit cost and requires panel modification, potentially limiting adoption in retrofit scenarios.

C. Software and Algorithm Enhancements

Software improvements could enhance system intelligence and user experience:

Machine learning occupancy prediction: Historical occupancy pattern analysis could enable predictive preconditioning, where the system anticipates occupancy based on learned patterns (e.g., typical work-from-home schedules, regular evening routines) and pre-activates lighting or climate control shortly before expected occupancy for improved comfort.

Adaptive threshold tuning: Implementing automatic calibration routines that adjust ultrasonic sensor thresholds based on observed environmental conditions would reduce installation complexity and improve long-term reliability as room configurations change due to furniture rearrangement or seasonal factors.

Energy disaggregation: Although the current implementation employs direct per-load metering, analyzing power signature patterns could identify specific appliance types and operating modes, providing more detailed consumption insights without additional hardware investment.

Advanced cost optimization: Supporting time-of-use electricity tariffs where available would enable load scheduling to minimize cost by deferring discretionary consumption to off-peak periods while maintaining comfort requirements.

D. User Experience Improvements

Enhanced interfaces and ecosystem integration would improve accessibility:

Native mobile applications: Developing iOS and Android applications would provide superior user experiences compared to web interfaces through platform-specific UI conventions, background operation, push notifications for alerts, and offline data caching. Development costs are higher but user satisfaction and engagement improve significantly with native applications.

Voice assistant integration: Connecting OptiWatt to popular voice assistants such as Google Assistant, Amazon Alexa, or Apple HomeKit would enable voice control and integration with existing smart home ecosystems. This requires implementing appropriate API protocols and potentially introducing cloud connectivity, which may compromise the current local-only architecture but significantly enhances convenience for users already invested in voice control ecosystems.

Visualization enhancements: Interactive consumption charts with time-range selection capabilities, comparative analysis across days, weeks, or months, and goal-setting with progress tracking would help users understand consumption patterns and motivate conservation behaviors. Export functionality supporting CSV or PDF report formats would enable external analysis and record-keeping.

Social features: Anonymous consumption comparison with similar households (matched by size, location, and season) could leverage social motivation for efficiency improvements. Gamification elements including achievements for conservation milestones and consistency streaks might encourage sustained engagement beyond initial deployment novelty.

Configuration wizards: Guided setup procedures with visual instructions, interactive component identification, and automated sensor calibration using test crossing sequences would reduce installation barriers for non-technical users and improve first-time success rates.

IX. Conclusion

This paper presented OptiWatt, a practical smart energy management system demonstrating how affordable microcontroller technology and straightforward algorithms can meaningfully address residential energy waste in developing regions. By combining real-time power monitoring via PZEM-004T modules with occupancy-aware automation using dual ultrasonic sensors, the system automatically reduces consumption in vacant rooms while preserving user control through comprehensive web-based manual override capabilities.

The implemented booth prototype validates the technical feasibility of the approach, showcasing a complete hardware and software system with professional enclosure design suitable for demonstration purposes and indicating pathways toward residential deployment. The modular architecture naturally supports future expansion to multi-room configurations, while the privacy-preserving sensor approach and local-only control architecture address key adoption barriers present in commercial systems relying on camera-based detection or mandatory cloud connectivity.

With total component costs under BDT 10,000, OptiWatt demonstrates economic viability for middle-income households in developing regions where commercial alternatives remain inaccessible due to cost barriers. While direct financial return on investment extends over many years due to relatively modest absolute savings in single-room applications, the system provides significant value through increased consumption awareness, billing transparency, convenience automation, and educational opportunities for understanding energy consumption patterns.

Key limitations include single-room scope in the current implementation, occasional occupancy counting errors during edge cases such as simultaneous bi-directional crossings or very rapid passages, and installation complexity requiring basic electrical knowledge for safe high-voltage connections. Future work should prioritize multi-room integration with centralized coordination, development of advanced analytics through machine learning approaches, custom PCB design for improved reliability and reduced manufacturing costs, and enhanced user interfaces including native mobile applications.

The OptiWatt prototype establishes a foundation for comprehensive residential energy optimization that balances efficiency, affordability, and user control without compromising privacy. Its modular design and open architecture invite continued development and customization, supporting evolution from demonstration system to practical deployment solution suitable for everyday residential applications in regions where smart home technology adoption currently faces economic and technical barriers.

Acknowledgment

The authors gratefully acknowledge Mr. Imran Hossain, Lecturer in the Department of Computer Science and Engineering at United International University, for his guidance, technical advice, and support throughout this project.

References

[1] S. Ahmed, "OptiWatt System Architecture Design," *Cirkit Designer*, 2025. [Online]. Available: https://app.cirkitdesigner.com/project/63a7375b-6e84-4add-9d03-7a2c141a902e. [Accessed Oct. 20, 2025].

[2] Espressif Systems, "ESP32 Series Datasheet," Version 3.9, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[3] Peacefair Electronics, "PZEM-004T V3.0 AC Digital Power Meter Module – Technical Manual," 2019. [Online]. Available: https://github.com/olehs/PZEM004T

[4] E. J. Morgan, "HC-SR04 Ultrasonic Sensor Datasheet," AllDatasheet.com, 2014. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/pdf/1132203/ETC2/HC-SR04.html

[5] A. K. Kashyap, R. Kumar, and S. Sharma, "Occupancy Detection Systems for Smart Buildings: A Review," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 4037–4051, Feb. 2021.

[6] G. W. Hart, "Nonintrusive Appliance Load Monitoring," *Proceedings of the IEEE*, vol. 80, no. 12, pp. 1870–1891, Dec. 1992.

[7] H. Sharif and A. Al-Turjman, "Smart Home Energy Management Systems: A Review," *Journal of Ambient Intelligence and Humanized Computing*, vol. 12, pp. 4357–4374, 2021.

[8] J. Han, C.-S. Choi, and I. Lee, "More Efficient Home Energy Management System Based on ZigBee Communication and Infrared Remote Controls," *IEEE Transactions on Consumer Electronics*, vol. 57, no. 1, pp. 85–89, Feb. 2011.

[9] U.S. Department of Energy, "Energy Saver Guide: Tips on Saving Money and Energy at Home," Office of Energy Efficiency & Renewable Energy, 2020.

Appendix A
Complete ESP32 Firmware

The full source code used for the OptiWatt system is included below. This firmware implements all core functionality including occupancy detection, energy monitoring, automation logic, and web server interface.

Listing 1: OptiWatt ESP32 Web Server Code

```
/*****************************************
 * OptiWatt Smart Energy Management System
 * ESP32 Web Server Implementation
 * Board : ESP32 DevKit V1
 * Course: EEE 2124 - Electronics Laboratory
 * Group : 07, Section: O
 *****************************************/

#include <WiFi.h>
#include <WebServer.h>
#include <PZEM004Tv30.h>
#include <Preferences.h>

/* ================== WIFI CONFIG
    ================== */
const char* ssid = "OptiWatt";
const char* password = "PasswordNai";
```

```
/* ================== EMBEDDED HTML / CSS / JS ================== */
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>  OptiWatt Smart Booth</title>
<style>
:root {
  --bg:#0f172a; --card:#1e293b; --text:#f1f5f9;
  --green:#22c55e; --red:#ef4444; --blue:#3b82f6; --muted:#94a3b8;
}
.light { --bg:#f8fafc; --card:#ffffff; --text:#1e293b; }
body {
  background:var(--bg); color:var(--text);
  font-family:Inter,Arial,sans-serif; margin:0; padding:0;
  transition:background .3s,color .3s;
}
header { text-align:center; padding:1rem 0; }
h1 {margin:0;font-size:1.8rem;}
h2 {margin:.2rem 0 .8rem;font-weight:400;font-size:1rem;}
.container {
  display:grid; gap:1rem;
  grid-template-columns:repeat(auto-fit,minmax(250px,1fr));
  padding:0 1rem 2rem;
}
.card {
  background:var(--card); border-radius:1rem;
  padding:1rem; box-shadow:0 2px 6px #0002;
}
button {
  border:none; border-radius:.5rem;
  padding:.5rem 1rem; font-weight:600; cursor:pointer;
  color:#fff; transition:opacity .2s;
}
button:disabled{opacity:.5;cursor:not-allowed;}
.btn-green{background:var(--green);}
.btn-red{background:var(--red);}
.btn-blue{background:var(--blue);width:100%;}
.toggle{position:fixed;top:.8rem;right:.8rem;cursor:pointer;font-size:1.4rem;}
.badge{padding:.2rem .6rem;border-radius:1rem;font-size:.8rem;}
.badge.on{background:var(--green);color:#fff;}
.badge.off{background:#64748b;color:#fff;}
.progress{height:8px;border-radius:4px;background:#475569;overflow:hidden;}
.progress span{display:block;height:100%;background:var(--green);width:0%;}
.warning{background:#facc15;color:#000;padding:.4rem;border-radius:.5rem;font-size:.8rem;margin-bottom:.5rem;display:none;}
footer{font-size:.8rem;padding:1rem 2rem;opacity:.7;}
.grid2x2{display:grid;grid-template-columns:1fr 1fr;gap:.3rem;}
.stat{font-size:.9rem;}
.small{color:var(--muted);font-size:.85rem}
.modal{position:fixed;inset:0;background:rgba(0,0,0,.35);display:none;align-items:center;justify-content:center;z-index:20}
.modal .box{background:var(--card);border:1px solid #334155;border-radius:12px;padding:16px;width:min(480px,92%)}
.modal .box h3{margin:.3rem 0}
.modal .actions{display:flex;gap:8px;justify-content:flex-end;margin-top:10px}
</style>
</head>
<body>
<div class="toggle" id="themeToggle"> </div>
<header>
  <h1>  OptiWatt Smart Booth</h1>
  <h2>Smart Energy Management System</h2>
  <div id="occBadge" class="badge off">Vacant</div>
  <button id="globalAutoBtn" class="btn-blue" style="margin-top:.5rem;">Global Auto: ON</button>
</header>

<section class="container" id="stats">
  <div class="card" id="totalPower"><h3>Total Power</h3><div id="totalPowerVal">0 W</div></div>
  <div class="card"><h3>Entries Today</h3><div id="entries">0</div></div>
  <div class="card"><h3>Exits Today</h3><div id="exits">0</div></div>
  <div class="card">
    <h3>Estimated Monthly Cost</h3>
    <div id="monthCost">0</div>
    <div class="small" id="projExplain">Avg-based projection from MTD energy</div>
  </div>
</section>

<section class="container">
  <div class="card" id="ledCard">
    <h3> LED <span id="ledAutoBadge" class="badge on">AUTO</span></h3>
    <div id="ledWarn" class="warning"> Physical switch active. Web controls disabled until released.</div>
    <div class="grid2x2">
      <div class="stat">V: <span id="ledV">0</span></div>
      <div class="stat">I: <span id="ledI">0</span></div>
      <div class="stat">P: <span id="ledP">0</span></div>
      <div class="stat">E: <span id="ledE">0</span> kWh</div>
    </div>
    <div style="margin-top:.6rem;">
      <div>Todays Cost (est.): <span id="ledCost">0</span></div>
      <div>Goal (daily): <span id="ledGoalVal">100</span> kWh <button id="ledGoalEdit"></button></div>
      <div class="progress"><span id="ledProg"></span></div>
    </div>
    <div style="margin-top:.8rem;">
      <button class="btn-green" id="ledOnBtn">Turn ON</button>
      <button class="btn-red" id="ledOffBtn">Turn OFF</button>
      <button class="btn-blue" id="ledAutoBtn">Toggle Automation</button>
    </div>
  </div>

  <div class="card" id="plugCard">
    <h3> Plug <span id="plugAutoBadge" class="badge on">AUTO</span></h3>
    <div id="plugWarn" class="warning"> Physical switch active. Web controls disabled until released.</div>
    <div class="grid2x2">
      <div class="stat">V: <span id="plugV">0</span></
```

```
        div>
    <div class="stat">I: <span id="plugI">0</span></
        div>
    <div class="stat">P: <span id="plugP">0</span></
        div>
    <div class="stat">E: <span id="plugE">0</span>
        kWh</div>
   </div>
   <div style="margin-top:.6rem;">
    <div>Todays Cost (est.): <span id="plugCost">0</
        span></div>
    <div>Goal (daily): <span id="plugGoalVal">100</
        span> kWh <button id="plugGoalEdit"></
        button></div>
    <div class="progress"><span id="plugProg"></span
        ></div>
   </div>
   <div style="margin-top:.8rem;">
    <button class="btn-green" id="plugOnBtn">Turn ON
        </button>
    <button class="btn-red" id="plugOffBtn">Turn OFF
        </button>
    <button class="btn-blue" id="plugAutoBtn">Toggle
         Automation</button>
   </div>
  </div>
</section>

<footer>
  <b> Technical Note: Presence Detection System</b><
      br>
  Two HC-SR04 ultrasonics: <b>Entry</b> = Door  4 cm
       then Inside  30 cm within 2 s; <b>Exit</b> =
      Inside then Door within 2 s.<br>
  Safeguards: median(9) sampling, 300 ms debounce,
      1.2 s refractory, 3 s vacancy off-delay, clamp
       occupancy  0.
</footer>

<!-- Modal -->
<div id="modal" class="modal" role="dialog" aria-
    modal="true">
  <div class="box">
    <h3>Automation is off for this appliance</h3>
    <p class="small">Enable automation again? If
        enabled, the device will follow occupancy and
         global rules.</p>
    <div class="actions">
     <button id="keepManual" class="btn-red">Keep
         Manual</button>
     <button id="enableAuto" class="btn-blue">Enable
         Automation</button>
    </div>
  </div>
</div>

<script>
/* ---------- THEME ---------- */
const ls = localStorage;
let lightMode = ls.getItem('theme')==='light';
if(lightMode) document.body.classList.add('light');
document.getElementById('themeToggle').onclick = ()
    =>{
  lightMode = !lightMode;
  document.body.classList.toggle('light');
  document.getElementById('themeToggle').textContent
       = lightMode ? '' : '';
  ls.setItem('theme', lightMode?'light':'dark');
};

/* ---------- HELPERS ---------- */
function daysInMonth(d){ return new Date(d.
    getFullYear(), d.getMonth()+1, 0).getDate(); }
function mKey(dev){ const d=new Date(); return `
    mBase_${dev}_${d.getFullYear()}_${d.getMonth()
    +1}`; }
function dKey(dev){ const d=new Date(); return `
    dBase_${dev}_${d.getFullYear()}_${d.getMonth()
    +1}_${d.getDate()}`; }
function ensureBase(key, curr){
  if(ls.getItem(key)===null) ls.setItem(key, String(
      curr));
  return parseFloat(ls.getItem(key));
}

/* Slabbed pricing (BD) */
function slabCost(kwh){
  kwh = Math.max(0, kwh);
  let remain=kwh,c=0;
  if(remain<=50) return remain*4.19;
  c+=50*4.19; remain-=50;
  if(remain<=25) return c+remain*5.72;
  c+=25*5.72; remain-=25;
  if(remain<=125) return c+remain*6.00;
  c+=125*6.00; remain-=125;
  if(remain<=100) return c+remain*6.34;
  c+=100*6.34; remain-=100;
  if(remain<=200) return c+remain*9.94;
  c+=200*9.94; remain-=200;
  return c + remain*11.46;
}

/* ---------- CONTROLS ---------- */
function api(path){ return fetch(path).then(()=>
    refresh()); }
function control(dev,action){ return api(`/control?
    device=${dev}&action=${action}`); }
document.getElementById('globalAutoBtn').onclick =
    ()=> api('/globalAuto');

document.getElementById('ledOnBtn').onclick = ()=>
    control('led','on').then(()=>showModal('led'));
document.getElementById('ledOffBtn').onclick = ()=>
    control('led','off').then(()=>showModal('led'));
document.getElementById('ledAutoBtn').onclick = ()=>
    control('led','auto');

document.getElementById('plugOnBtn').onclick = ()=>
    control('plug','on').then(()=>showModal('plug'))
    ;
document.getElementById('plugOffBtn').onclick = ()=>
    control('plug','off').then(()=>showModal('plug')
    );
document.getElementById('plugAutoBtn').onclick = ()
    =>control('plug','auto');

document.getElementById('ledGoalEdit').onclick = ()
    =>{
  const v = prompt("LED daily goal (kWh):", document
      .getElementById('ledGoalVal').textContent);
  if(v) api(`/setGoal?device=led&goal=${v}`);
};
document.getElementById('plugGoalEdit').onclick = ()
    =>{
  const v = prompt("Plug daily goal (kWh):",
      document.getElementById('plugGoalVal').
      textContent);
  if(v) api(`/setGoal?device=plug&goal=${v}`);
};

/* ---------- MODAL ---------- */
let modalDev = null;
function showModal(dev){ modalDev=dev; document.
    getElementById('modal').style.display='flex'; }
function closeModal(){ modalDev=null; document.
    getElementById('modal').style.display='none'; }
document.getElementById('keepManual').onclick =
    closeModal;
```

```
document.getElementById('enableAuto').onclick = () =>{
  if(!modalDev) return closeModal();
  api(`/control?device=${modalDev}&action=auto`).then(closeModal);
};

/* ---------- UI UPDATE ---------- */
function setSwitchDisabled(dev, active){
  document.getElementById(dev+'Warn').style.display = active ? 'block' : 'none';
  document.getElementById(dev+'OnBtn').disabled = active;
  document.getElementById(dev+'OffBtn').disabled = active;
  document.getElementById(dev+'AutoBtn').disabled = active;
}

const lastAuto = { led:true, plug:true };

function updateDev(name,obj){
  document.getElementById(name+'V').textContent = obj.V.toFixed(1);
  document.getElementById(name+'I').textContent = obj.I.toFixed(3);
  document.getElementById(name+'P').textContent = obj.P.toFixed(1);
  document.getElementById(name+'E').textContent = obj.E.toFixed(3);

  setSwitchDisabled(name, !!obj.switch);

  const b = document.getElementById(name+'AutoBadge');
  b.textContent = obj.auto ? 'AUTO' : 'MANUAL';
  b.className = 'badge ' + (obj.auto?'on':'off');

  // daily goal progress from today's baseline
  const dBase = ensureBase(dKey(name), obj.E);
  const dayE = Math.max(0, obj.E - dBase);
  const prog = Math.min(100, (dayE / Math.max(0.001, obj.goal)) * 100);
  document.getElementById(name+'GoalVal').textContent = obj.goal.toFixed(1);
  document.getElementById(name+'Prog').style.width = prog.toFixed(1)+'%';

  // show modal once when auto flips to OFF (manual action)
  if(lastAuto[name] && !obj.auto) showModal(name);
  lastAuto[name] = !!obj.auto;

  // rough per-device today's cost using average rate (device-level slab is non-linear)
  const avgRate = 7.742;
  document.getElementById(name+'Cost').textContent = (dayE * avgRate).toFixed(0);
}

function refresh(){
  fetch('/state',{cache:'no-store'})
    .then(r=>r.json())
    .then(d=>{
      const occ = document.getElementById('occBadge');
      occ.textContent = d.occupied ? 'Occupied' : 'Vacant';
      occ.className = 'badge ' + (d.occupied?'on':'off');

      document.getElementById('entries').textContent = d.entryCount;
      document.getElementById('exits').textContent = d.exitCount;
      document.getElementById('globalAutoBtn').textContent = "Global Auto: " + (d.globalAuto?"ON":"OFF");

      updateDev('led', d.led);
      updateDev('plug', d.plug);

      const totalP = (d.led.P + d.plug.P).toFixed(1);
      document.getElementById('totalPowerVal').textContent = `${totalP} W`;

      // MTD baseline & average-based projection (applied to combined energy)
      const mBaseLed = ensureBase(mKey('led'), d.led.E);
      const mBasePlug = ensureBase(mKey('plug'), d.plug.E);
      const mtdKwh = Math.max(0, (d.led.E - mBaseLed) + (d.plug.E - mBasePlug));
      const costMTD = slabCost(mtdKwh);

      const now = new Date();
      const daysElapsed = Math.max(1, now.getDate());
      const totalDays = daysInMonth(now);
      const avgDaily = costMTD / daysElapsed;
      const projected = avgDaily * totalDays;

      document.getElementById('monthCost').textContent = "" + projected.toFixed(0);
      document.getElementById('projExplain').textContent =
        `MTD kWh: ${mtdKwh.toFixed(2)}  Cost MTD: ${costMTD.toFixed(0)}  Avg/day: ${avgDaily.toFixed(0)}  Days: ${daysElapsed}/${totalDays}`;
    })
    .catch(console.error);
}
setInterval(refresh, 1000);
refresh();
</script>
</body>
</html>
)rawliteral";

/* ================== PIN DEFINITIONS ================== */
#define RELAY_LED  14
#define RELAY_PLUG 26
#define SWITCH_LED 25
#define SWITCH_PLUG 33
#define DOOR_TRIG  5
#define DOOR_ECHO  18
#define INSIDE_TRIG 4
#define INSIDE_ECHO 19
#define RELAY_ACTIVE_LOW false

/* ================== OCCUPANCY STATE MACHINE ================== */
enum OccState : uint8_t { VACANT=0, ENTRY_ARMED, OCCUPIED, EXIT_ARMED };

/* ================== STRUCTS ================== */
struct Device {
  const char* name;
  uint8_t relayPin;
  uint8_t switchPin;
  bool relayState;    // current relay output
  bool switchState;   // HIGH idle, LOW pressed
  bool autoEnabled;   // per-device automation
  float V, I, P, E, goal; // telemetry + goal
  // per-device vacancy off delay (3s)
  bool offDelayPending;
```

```
  unsigned long offDelayStartMs;
};

Device led = {"led", RELAY_LED, SWITCH_LED, false, HIGH, true, 0,0,0,0,100, false, 0};
Device plug = {"plug", RELAY_PLUG, SWITCH_PLUG, false, HIGH, true, 0,0,0,0,100, false, 0};

/* ================== GLOBALS ================== */
bool globalAuto = true;

volatile OccState occState = VACANT;
volatile unsigned long occStateSince = 0;
volatile int occupancy = 0;
int entryCount = 0;
int exitCount = 0;

/* timing (ms) */
const unsigned long REFRACTORY_MS = 1200UL;
const unsigned long WINDOW_PAIR_MS = 2000UL;
const unsigned long VACANCY_DELAY_MS= 3000UL;
const unsigned long DEBOUNCE_MS = 300UL;

/* ultrasonic schedule */
const uint32_t TASK_ULTRA_PERIOD_MS = 40;
const uint32_t ULTRA_TIMEOUT_US = 12000; // pulseIn timeout

/* PZEM & state poll periods */
const uint32_t PZEM_PERIOD_MS = 500;

/* thresholds */
const float DOOR_THRESH_CM = 4.0f;
const float INSIDE_THRESH_CM = 30.0f;

/* ring buffers for median(9) */
static const int MED_N = 9;
float doorBuf[MED_N]; int doorIdx=0; bool doorFilled=false;
float inBuf[MED_N]; int inIdx=0; bool inFilled=false;

/* trigger times */
unsigned long lastDoorEventMs=0, lastInsideEventMs=0;
unsigned long refractoryUntilMs=0;
unsigned long armWindowUntilMs=0;
unsigned long vacantSinceMs=0;

/* ================== SERVICES ================== */
Preferences prefs;
WebServer server(80);
HardwareSerial pzemSerial(2); // UART2

/* PZEM devices (shared UART2 RX=16 TX=17, distinct modbus addresses) */
PZEM004Tv30 pzemLed (pzemSerial, 16, 17, 0x02);
PZEM004Tv30 pzemPlug(pzemSerial, 16, 17, 0x03);

/* ================== HELPERS ================== */
static inline float safeVal(float v){ return isfinite(v)? v : 0.0f; }

void setRelay(Device &d, bool on){
  digitalWrite(d.relayPin, RELAY_ACTIVE_LOW ? !on : on);
  d.relayState = on;
}

void pushDoor(float v){ doorBuf[doorIdx++] = v; if(doorIdx>=MED_N){doorIdx=0;doorFilled=true;} }
void pushInside(float v){ inBuf[inIdx++] = v; if(inIdx>=MED_N){inIdx=0;inFilled=true;} }

float medianFrom(float *buf, int n){
  float tmp[MED_N];
  for(int i=0;i<n;i++) tmp[i]=buf[i];
  // insertion sort for n<=9
  for(int i=1;i<n;i++){
    float k=tmp[i]; int j=i-1;
    while(j>=0 && tmp[j]>k){ tmp[j+1]=tmp[j]; j--; }
    tmp[j+1]=k;
  }
  return tmp[n/2];
}

float medianDoor(){
  int n = doorFilled ? MED_N : (doorIdx==0?1:doorIdx);
  return medianFrom(doorBuf, n);
}
float medianInside(){
  int n = inFilled ? MED_N : (inIdx==0?1:inIdx);
  return medianFrom(inBuf, n);
}

float readUltrasonicCM(uint8_t trig, uint8_t echo){
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);
  unsigned long dur = pulseIn(echo, HIGH, ULTRA_TIMEOUT_US);
  if(!dur) return 9999.0f;
  float cm = dur / 58.2f;
  return cm>0? cm : 9999.0f;
}

/* ================== OCCUPANCY LOGIC ================== */
inline bool within(uint32_t now, uint32_t until){ return (int32_t)(until - now) > 0; }

void setOccState(OccState s){ occState = s; occStateSince = millis(); }

void confirmEntry(){
  if(millis() < refractoryUntilMs) return;
  occupancy++; if(occupancy<0) occupancy=0;
  entryCount++;
  setOccState(OCCUPIED);
  refractoryUntilMs = millis() + REFRACTORY_MS;
  // auto actions (global + per device)
  if(globalAuto){
    if(led.autoEnabled) setRelay(led, true);
    if(plug.autoEnabled) setRelay(plug, true);
    led.offDelayPending = plug.offDelayPending = false;
  }
}

void confirmExit(){
  if(millis() < refractoryUntilMs) return;
  if(occupancy>0) occupancy--;
  exitCount++;
  if(occupancy==0){
    setOccState(VACANT);
    vacantSinceMs = millis();
  }else{
    setOccState(OCCUPIED);
  }
  refractoryUntilMs = millis() + REFRACTORY_MS;
}

void processOccupancy(float doorCM, float insideCM){
  uint32_t now = millis();
```

```
  bool doorTrig = (doorCM <= DOOR_THRESH_CM);
  bool insideTrig = (insideCM <= INSIDE_THRESH_CM);

  // per-sensor debounce & refractory
  if(doorTrig && (now - lastDoorEventMs) < DEBOUNCE_MS) doorTrig = false;
  if(insideTrig && (now - lastInsideEventMs) < DEBOUNCE_MS) insideTrig = false;
  if(now < refractoryUntilMs){ doorTrig=false; insideTrig=false; }

  switch(occState){
    case VACANT:
      if(doorTrig){ lastDoorEventMs = now; setOccState(ENTRY_ARMED); armWindowUntilMs = now + WINDOW_PAIR_MS; }
      break;
    case ENTRY_ARMED:
      if(!within(now, armWindowUntilMs)) setOccState(VACANT);
      else if(insideTrig){ lastInsideEventMs = now; confirmEntry(); }
      break;
    case OCCUPIED:
      if(insideTrig){ lastInsideEventMs = now; setOccState(EXIT_ARMED); armWindowUntilMs = now + WINDOW_PAIR_MS; }
      break;
    case EXIT_ARMED:
      if(!within(now, armWindowUntilMs)) setOccState(OCCUPIED);
      else if(doorTrig){ lastDoorEventMs = now; confirmExit(); }
      break;
  }

  // per-device 3s delay after vacancy
  if(occupancy==0 && globalAuto){
    if(vacantSinceMs && (now - vacantSinceMs) >= VACANCY_DELAY_MS){
      if(led.autoEnabled) setRelay(led,false);
      if(plug.autoEnabled) setRelay(plug,false);
      led.offDelayPending = plug.offDelayPending = false;
      vacantSinceMs = 0;
    }
  }
}

/* ================== SWITCHES & RELAYS ================== */
void handlePhysicalSwitch(Device &d){
  bool state = digitalRead(d.switchPin);
  if(state != d.switchState){
    delay(30); // debounce
    if(digitalRead(d.switchPin)==state){
      d.switchState = state;
      if(state==LOW){ // press -> toggle & disable auto
        setRelay(d, !d.relayState);
        if(d.autoEnabled){
          d.autoEnabled=false; savePrefs();
        }
      }
    }
  }
}

void updateRelays(){
  if(!globalAuto) return;

  const bool shouldOn = (occupancy>0);
  Device* arr[2] = { &led, &plug };

  for(Device* d: arr){
    if(!d->autoEnabled){ d->offDelayPending=false; continue; }

    if(shouldOn){
      if(!d->relayState) setRelay(*d, true);
      d->offDelayPending=false;
    }else{
      if(d->relayState){
        if(!d->offDelayPending){
          d->offDelayPending = true;
          d->offDelayStartMs = millis();
        }else if(millis() - d->offDelayStartMs >= VACANCY_DELAY_MS){
          setRelay(*d, false);
          d->offDelayPending = false;
        }
      }else{
        d->offDelayPending = false;
      }
    }
  }
}

/* ================== TELEMETRY & COST ================== */
void updateTelemetry(){
  led.V = safeVal(pzemLed.voltage());
  led.I = safeVal(pzemLed.current());
  led.P = safeVal(pzemLed.power());
  led.E = safeVal(pzemLed.energy());

  plug.V = safeVal(pzemPlug.voltage());
  plug.I = safeVal(pzemPlug.current());
  plug.P = safeVal(pzemPlug.power());
  plug.E = safeVal(pzemPlug.energy());
}

/* ================== STORAGE ================== */
void loadPrefs(){
  prefs.begin("OptiWatt", true);
  globalAuto   = prefs.getBool("globalAuto", true);
  led.autoEnabled = prefs.getBool("ledAuto", true);
  plug.autoEnabled = prefs.getBool("plugAuto", true);
  led.goal     = prefs.getFloat("ledGoal", 100.0f);
  plug.goal    = prefs.getFloat("plugGoal", 100.0f);
  prefs.end();
}

void savePrefs(){
  prefs.begin("OptiWatt", false);
  prefs.putBool("globalAuto", globalAuto);
  prefs.putBool("ledAuto", led.autoEnabled);
  prefs.putBool("plugAuto", plug.autoEnabled);
  prefs.putFloat("ledGoal", led.goal);
  prefs.putFloat("plugGoal", plug.goal);
  prefs.end();
}

/* ================== WEB API ================== */
String jsonState(){
  bool ledSwitchActive = (led.switchState == LOW);
  bool plugSwitchActive = (plug.switchState == LOW);

  char buf[1024];
  snprintf(buf, sizeof(buf),
    "{\"led\":{\"V\":%.2f,\"I\":%.3f,\"P\":%.1f,\"E\":%.3f,\"relay\":%s,\"switch\":%s,\"auto\":%s,\"goal\":%.1f},"
    "\"plug\":{\"V\":%.2f,\"I\":%.3f,\"P\":%.1f,\"E\":%.3f,\"relay\":%s,\"switch\":%s,\"auto\":%
```

```
        s,\"goal\":%.1f},"
    "\"occupied\":%s,\"globalAuto\":%s,\"entryCount\":%d,\"exitCount\":%d}",
    led.V,led.I,led.P,led.E, led.relayState?"true":"false", ledSwitchActive ?"true":"false", led.autoEnabled ?"true":"false", led.goal,
    plug.V,plug.I,plug.P,plug.E, plug.relayState?"true":"false", plugSwitchActive?"true":"false", plug.autoEnabled?"true":"false", plug.goal,
    (occupancy>0)?"true":"false", globalAuto?"true":"false", entryCount, exitCount);
  return String(buf);
}

void handleState(){ server.send(200,"application/json",jsonState()); }

void handleControl(){
  if(!server.hasArg("device") || !server.hasArg("action")){
    server.send(400,"text/plain","Bad request"); return;
  }
  String dev = server.arg("device");
  String act = server.arg("action");

  Device* d = nullptr;
  if    (dev=="led") d = &led;
  else if(dev=="plug") d = &plug;
  else { server.send(404,"text/plain","Unknown device"); return; }

  if(act=="on"){
    setRelay(*d,true);
    if(d->autoEnabled){ d->autoEnabled=false; savePrefs(); }
  } else if(act=="off"){
    setRelay(*d,false);
    if(d->autoEnabled){ d->autoEnabled=false; savePrefs(); }
  } else if(act=="auto"){
    bool prev = d->autoEnabled;
    d->autoEnabled = !d->autoEnabled;
    savePrefs();
    if(d->autoEnabled && globalAuto){
      // conform to occupancy immediately
      setRelay(*d, occupancy>0);
      d->offDelayPending=false;
    }
    if(prev && !d->autoEnabled){ d->offDelayPending=false; }
  } else {
    server.send(400,"text/plain","Unknown action"); return;
  }

  server.send(200,"text/plain","OK");
}

void handleGlobalAuto(){
  globalAuto = !globalAuto;
  if(globalAuto){
    if(led.autoEnabled) setRelay(led, occupancy>0);
    if(plug.autoEnabled) setRelay(plug, occupancy>0);
    led.offDelayPending = plug.offDelayPending = false;
  }
  savePrefs();
  server.send(200,"text/plain","OK");
}

void handleSetGoal(){
  if(!server.hasArg("device") || !server.hasArg("goal")){
    server.send(400,"text/plain","Bad request"); return;
  }
  String dev = server.arg("device");
  float goal = server.arg("goal").toFloat();
  if(goal<=0){ server.send(400,"text/plain","Invalid goal"); return; }

  if(dev=="led") led.goal = goal;
  else if(dev=="plug")plug.goal = goal;
  else { server.send(404,"text/plain","Unknown device"); return; }

  savePrefs();
  server.send(200,"text/plain","OK");
}

/* ================== SCHEDULER ================== */
uint32_t tUltraMs=0, tPzemMs=0;

void setup(){
  Serial.begin(115200);

  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while(WiFi.status()!=WL_CONNECTED){ delay(300); Serial.print("."); }
  Serial.println("\nWiFi connected: "+WiFi.localIP().toString());

  // PZEM UART2
  pzemSerial.begin(9600, SERIAL_8N1, 16, 17);

  pinMode(RELAY_LED, OUTPUT);
  pinMode(RELAY_PLUG, OUTPUT);
  pinMode(SWITCH_LED, INPUT_PULLUP);
  pinMode(SWITCH_PLUG, INPUT_PULLUP);
  pinMode(DOOR_TRIG, OUTPUT);
  pinMode(DOOR_ECHO, INPUT);
  pinMode(INSIDE_TRIG, OUTPUT);
  pinMode(INSIDE_ECHO, INPUT);

  // init relays OFF
  setRelay(led,false);
  setRelay(plug,false);

  // initial switch snapshot
  led.switchState = digitalRead(SWITCH_LED);
  plug.switchState = digitalRead(SWITCH_PLUG);

  loadPrefs();

  // API routes
  server.on("/state", handleState);
  server.on("/control", handleControl);
  server.on("/globalAuto", handleGlobalAuto);
  server.on("/setGoal", handleSetGoal);

  // root UI
  server.on("/", [](){ server.send_P(200, "text/html", index_html); });

  // 302 -> root for unknown paths
  server.onNotFound([](){
    server.sendHeader("Location", "/");
    server.send(302, "text/plain", "");
  });

  server.begin();
  Serial.println("HTTP server started");

  occStateSince = millis();
```

```
  tUltraMs = tPzemMs = millis();
}

void loop(){
  server.handleClient();

  // physical switches
  handlePhysicalSwitch(led);
  handlePhysicalSwitch(plug);

  // periodic ultrasonic sampling -> ring buffers -> state machine
  uint32_t now = millis();
  if(now - tUltraMs >= TASK_ULTRA_PERIOD_MS){
    tUltraMs = now;
    float d = readUltrasonicCM(DOOR_TRIG, DOOR_ECHO);
    float i = readUltrasonicCM(INSIDE_TRIG, INSIDE_ECHO);
    pushDoor(d);
    pushInside(i);
    processOccupancy(medianDoor(), medianInside());
  }

  // per-device automation enforcement (and vacancy off-delay)
  updateRelays();

  // telemetry each ~500ms
  if(now - tPzemMs >= PZEM_PERIOD_MS){
    tPzemMs = now;
    updateTelemetry();
  }

  // brief yield to keep loop responsive
  delay(10);
}
```